Promo Finder

Problem 1: Find the Promos | Kuku AI Generalist assignment

Problem and scope

Editors repeat the same search for every new show. I chose Problem 1 because cut points and supporting text can be checked against the screenplay. This prototype searches all episodes and produces an editor review queue. It does not edit footage or predict ad performance.

Assumptions

- Times are episode-relative; cuts are [start, end), using exact source labels. Simultaneous camera, action, sound and dialogue tracks do not add runtime.
- A complete spoken hook must fit inside the first three seconds. Visual-action prose gives weaker timing evidence and needs footage review.
- All episodes are searched; the bounded shortlist can miss good moments. Continuous clips are the default. Same-episode stitches are experimental; cross-episode joins are not generated.
- The source pack, transcripts and raw requests remain private. Publish code and synthetic examples only. An editor approves every final cut.

Method and model boundary

I considered rule-only windows, whole-episode model proposals, and a hybrid. Rules are cheap but miss meaning; proposals risk invented timecodes. The hybrid keeps cuts in code and uses models for editorial judgment.

The parser read 50 files, 100 scenes and 3,727 timed blocks. Code enumerates valid 20-60 second windows, rejects cuts through speech, retrieves tension plus late unresolved beats, and builds optional ordered stitches. Refined retrieval produced 27,342 continuous windows and shortlisted 181 candidates, including 31 stitches.

A local Qwen3 4B model judges the clip alone. A separate call scores show-premise fit; a fresh clip-only critic checks the pre-ranked top 30. Only reviewed clips with unresolved status and hook, cliffhanger and clarity each at least 3/5 enter the final queue. Code validates source times, duration, order and exact hook/end excerpts.

Ranking uses hook 28%, cliffhanger 26%, clarity 20%, emotion 10%, premise 10%, minus spoiler risk 6%. Repetition penalties encourage varied types and episodes; overlapping alternatives are suppressed. Every clip includes its scores, risks and segment list. Provider config swaps local, Gemini and compatible API endpoints; offline mode is clearly a rule proxy.

What I would review first

The tool ranks Episode 12 first, but its opening question is generic and its ending is a defiant reaction. My first editorial test would be its second-ranked clip: Episode 44, 00:49.20-01:10.40, 21.2 seconds. The status rejection finishes in 1.8 seconds and the romantic rivalry is clear. Its final reaction is still a weaker cliffhanger than an imminent unanswered action. Check footage before posting. This editorial disagreement is preserved beside the raw ranking.

Evidence and limits

What the actual runs establish

The final queue has 21 clips across 21 episodes. All 21 pass exact timestamp, 20-60 second duration, ordered segments, quote, content-range, speech-cut and opening-evidence checks. The 181-candidate shortlist also passes. There are zero exact or suppressed near-overlap duplicates in the final queue; its top 10 cover four types. All 79 tests pass. These are engineering checks, not editorial accuracy.

A fresh sampling stress run changed temperature/seed; 9/10 top picks matched and the first pick stayed the same. This tests sampling sensitivity, not identical-setting nondeterminism. Smaller-model comparisons also changed context policy and are confounded. The initial rule baseline, failed model runs and later refinements are retained unedited.

On one synthetic clip, Gemini in Antigravity scored clarity 4/5 versus Qwen's 3/5. Both scored the cliffhanger 5/5 and marked it unresolved. Gemini avoided Qwen's unsupported claims about the child and envelope but assumed the stranger was uninvited. The exact Gemini model and thinking level are unreported.

Experiments exposed useful failures. A tiny model copied example scores. Models borrowed opening-episode facts for later clips, so I separated premise scoring. A critic initially promoted unchecked clips indirectly; I gated ranking on review. Nine model-resolved clips still survived a penalty, so the final gate excludes them. A literal role-word repair changed an incorrect brother into an incorrect sister; source-review feedback was needed. The narrow guard cannot prove a rationale true.

Remaining weak picks include vague opening questions, cuts on a reaction or unfinished clause, joins that remove relationship context, and action descriptions whose full event may not occur before the cut. Only 0.1 seconds of one branding-action interval lies before its cut. Text cannot establish acting, audio, shot impact, exact sub-action timing or conversion.

Human review and measurement

Label all 21 final clips in the blank review CSV, about 30-40 minutes. Mark good/okay/bad and a reason; the script computes strict good fraction and precision@10 after all top 10 are labelled. No labels are prefilled and no precision claim is made. The private failure analysis gives seven questionable picks and three missed boundary choices. An editor reviews the top 10 against footage, approves starts/ends and checks spoilers. A live ad test is needed for retention or acquisition results.

Scale and cost

The main path plus recorded refinements made 397 actual local requests: 272,055 input and 22,000 output tokens, averaging 5,441/440 per episode. Inference API spend was $0; power and hardware are excluded. The original full model pass took 13.9 minutes on this laptop; later reports reused cached responses, not new inference.

At rates checked on 5 November 2026, Gemini 3.1 Pro costs an estimated $0.81 per 50-episode show ($0.016/episode) using Qwen token counts and $2/$12 per million input/output tokens. Tokenization, reasoning and retries can change the estimate. Live Gemini API testing remains pending. See Google API pricing for the rates.

Weekly operation is one private batch per show, an expected episode count, source hashes, cached judgments and bounded transport/schema retries. Current inference is sequential. A multi-show worker queue and per-episode parse/summary cache are next steps, not implemented claims. Keep the editor at the top-10 video review and boundary-approval steps.

AI use and submission evidence

Codex helped write code, tests, documents and the source audit. Local Qwen models produced the judgments for the 50 episodes. The private outputs folder contains the raw runs, model requests, failure analysis and review metrics. The public bundle includes the synthetic Gemini response and comparison.